PROGRAMACION III - Trabajo final

Introducción

En el siguiente documento se encuentra la consigna para el trabajo final de la materia Programación III. Constará de un sistema para manejar tu cartera de criptomonedas, y realizar cálculos con la misma.

Es posible modificar cierto comportamiento de este sistema. Por ejemplo, en vez de crear dos pantallas (una para compra de criptomonedas y otra para venta), puede ser una sola en donde se tenga que seleccionar si es compra o venta con un select. Se da cierta libertad al alumno.

El trabajo es individual, de a dos como máximo.

Introducción al dominio

Las criptomonedas son un tipo de monedas virtuales que se pueden intercambiar entre los usuarios. La más conocida es Bitcoin, aunque existen muchísimas más (en este link puedes ver una lista de las más conocidas).

Éstas tienen un valor, que puede ser estable (por ejemplo, la criptomoneda USDC siempre tiene un valor cercano a 1 dólar), o volátil (por ejemplo, el precio del bitcoin varía constantemente).

Asimismo existen los “exchange”, que son plataformas en donde uno puede comprar y vender estas criptomonedas, y que el precio de compra y venta varía entre las mismas (se puede establecer un paralelismo con el dólar, en donde los “exchange” serían los bancos, los cuales en cada uno tiene un precio diferente -algunos más caros y otros más baratos-). Ejemplos de exchanges de Argentina son SatoshiTango, BuenBit, etc.

Tecnologías a usar

El alumno deberá crear tanto el frontend como el backend del sistema que se detalla a continuación.

Las tecnologías a usar serán:

- Frontend: Vue.js versión 3 o JavaScript
- Backend: ASP.NET
- Base de datos: cualquier motor SQL

El alumno podrá usar tecnologías adicionales (como por ejemplo Tailwaind, Docker, etc) en caso que lo sienta necesario.

Estructura de endpoints y DB

Aunque en el enunciado se determina una estructura de ejemplo para usar en los endpoints, no es necesario cumplirla al 100%.

Lo único importante a cumplir es:

- Que respete la funcionalidad pedida.
- Que use el método HTTP correcto (GET si obtiene los datos de un recurso, POST si crea uno, etc).

Servicios externos

Además de la API que el alumno construirá, deberá usar servicios externos para obtener el precio de la criptomoneda en los diferentes exchanges.

La mismas consultas se harán a la API de la página https://criptoya.com/. Para ver la documentación de la misma ingresar a https://criptoya.com/api (importante hacerlo para entender muchas cosas). Con respecto al exchange a usar, puede ser cualquiera (por ejemplo, en este documento uso SatoshiTango, aunque hay muchos más). Asimismo se puede usar otra API para obtener el precio en ARS (Pesos Argentinos) de las criptomonedas.

IMPORTANTE: aunque la documentación de la API de criptoya menciona que se debe pasar el volumen, la misma siempre devuelve el valor de una unidad. Por ejemplo, si consultamos a https://criptoya.com/api/satoshitango/usdc/ars/0.5, el valor devuelto será el valor de 1 USDC (no de 0.5).

Con respecto a las criptomonedas, debido a que existen muchas, la aplicación debe manejar al menos 3 (por ejemplo, Bitcoin, USDC y Ethereum), aunque pueden ser más (lo cual es mejor). La elección de estas 3 está a cargo del estudiante.

Pantalla dar de alta una nueva compra

Frontend

Una pantalla debe permitir dar de alta una nueva compra de criptomonedas. Para ello se debe ingresar:

- La criptomoneda que se compró (recordar que deben ser al menos 3 las opciones).
- La cantidad que se compró. Tener en cuenta de que este es un número decimal, ya que se pueden comprar porciones de criptomonedas (por ejemplo, en el caso del Bitcoin, al ser tan caro, casi siempre es menor a 1, como “0.00070”).
- La fecha y hora en la que se compró.

Una vez ingresado, validar que los datos sean correctos (por ejemplo, que la cantidad de criptomonedas sea mayor a 0).

Backend

Genera un endpoint que reciba una llamada POST y guarde los datos en la Base de Datos (y que no se pierda si se recarga la página).

Por ejemplo, podría ser un endpoint "*/transactions/*" del cuerpo que puedes usar:

```
{
   "crypto_code": "usdc",
   "action": "purchase",
   "crypto_amount": "1.01",
   "datetime": "2021-07-11 17:50"
}
```

Como se puede ver:

- crypto_code: el código de la criptomoneda, en minúsculas. Por ejemplo, para el Bitcoin será "bitcoin", para USDC "usdc", etc. Lo importante es elegir un código único para cada criptomoneda y respetarlo.
- action: indica si es una compra o venta. En este caso que estamos comprando, será "purchase". Cuando se da de alta una venta el valor es "sale"
- crypto_amount: la cantidad de criptomoneda que se compró.
- money: cuánto se pagó en Pesos Argentinos
- date: la fecha cuando se compró. En este caso se usa el formato "YYYY-MM-DD hh:ss".

Sin embargo, como se puede ver, no estamos guardando el monto que gastamos en total. Por ende, cuando damos de alta una transacción:

- Debemos realizar las validaciones
- Llamamos a la API de criptoya para ver cual es el precio de esa criptomoneda en ese momento, y lo multiplicamos por la cantidad ingresada.
- En la base de datos, además de guardar los datos que recibimos desde el frontend, guardamos el monto gastado en total
- En el ejemplo mostrado, debido a que compramos 1.01 y si el precio está a 1000 ARS, entonces el monto gastado sería "1.01 * 1000 = 1010"

Una vez hecho todo esto, debemos guardar en la base de datos esta transacción.

Pantalla dar de alta una nueva venta

Frontend

Asimismo como hay una pantalla para indicar una compra, debe haber una para que el usuario indique que vendió una cantidad X de criptomonedas. Para eso, la pantalla debe permitir ingresar:

- La criptomoneda que se vendió.
- La cantidad que se vendió.
- La fecha y hora.
- Cuánto se cobró.

Una vez ingresado, validar que los datos sean correctos.

Backend

Genera un endpoint para esta vista, o reusa el anterior. Por ejemplo, es un POST al endpoint "*/transactions/*" con el siguiente cuerpo:

```
{
   "crypto_code": "usdc",
   "action": "sale",
   "crypto_amount": "1.01",
   "datetime": "19-07-2021 20:50"
}
```

Notar que es muy similar a la request anterior (incluso en que debe calcular en el backend el monto que gastó), excepto que "action" cambia para indicar que fue una venta (*sale*).

Este endpoint, además de verificar que los datos sean válidos, debe validar que NO se pueda vender criptomonedas que no se tengan. Por ejemplo, si compré 2 USDC y luego intento vender 3 USDC, no debería permitirmelo.

Historial de movimientos

Frontend

Necesitamos otra pantalla en donde se puedan mostrar todos los movimientos de criptomonedas (es decir, todas las compras y ventas que hicimos),y su respectiva información.

Mostrar esta información en una tabla, en donde cada registro tendrá un botón para poder llevar a la lectura, edición y borrado de cada transacción.

Backend

Agregar un endpoint GET que permita obtener todas las transacciones (tanto de compra como de venta).

Por ejemplo, se podría agregar un GET a "/transactions" qué devuelta las compras y ventas, ordenadas por fecha de creación, y devolver el siguiente cuerpo:

```
[
   {
       "id": "1",
       "action": "sale",
       "crypto_code": "usdc",
       "crypto_amount": "1.01",
       "money": "170.98",
       "action": "sale",
       "datetime": "2021-11-07T20:50:00.000Z"
   },
   {
```

```
        "id": "2",
        "action": "purchase",
        "crypto_code": "usdc",
        "crypto_amount": "1.01",
        "money": "165.23",
        "action": "purchase",
        "datetime": "2021-11-07T17:50:00.000Z"
    }
]
```


Como se puede ver, este endpoint devuelve el monto que se gastó/recaudó en cada transacción bajo el atributo “money”.

Lectura, edición y borrado de las compras/ventas

Frontend

Asimismo como permitimos dar de alta una compra o venta, también debemos permitir poder leerla, modificarla o borrarla.

Un ejemplo de cómo implementar sería en la tabla donde se muestra el historial de movimientos:

- Tiene un botón “Ver”, que lo llevará a una vista en donde podrá ver toda la información de esa transacción.
- Tiene un botón “Editar”, que lo llevará a una vista en donde podrá editar toda la información de esa transacción.
- Tiene un botón “Borrar”, que abrirá un modal para que el usuario confirme si quiere borrar.

Backend

Supongamos que una transacción tiene el ID “1” (este ID se obtiene de cuando hacemos una request a “/transactions”):

- Para consultar la información de una transacción que la usaremos en la vista de la transacción o en la vista de edición (para autocompletar los campos), se debería llamar con GET a “*/transactions/{id}*” (por ejemplo, “*/transactions/1*”). Esto devolvería un cuerpo como el siguiente:


```
{
    "_id": "60eb148da4666761000216f9",
    "crypto_code": "usdc",
    "crypto_amount": "1.01",
    "money": "165.23",
    "action": "purchase",
    "datetime": "2021-11-07T17:50:00.000Z"
}
```

- Para editar la información de una transacción, se debe llamar con PATCH a "*/transactions/{id}*" y en el cuerpo un JSON con los nuevos valores. Si por ejemplo queremos actualizar el monto gastado de la transacción con id 1 a 170.55, entonces deberíamos llamar a "*/transactions/1*" e incluir un cuerpo como el siguiente:

```
{
   "money": "170.55"
}
```

- Por último, para eliminar una transacción, usar DELETE a la url "/transactions/{id}". Siguiendo con el ejemplo, si llamamos a "/transactions/1" con DELETE, se borra esa fila.

Aclaración: en la edición no es necesario calcular el monto total gastado ni validar que tenga suficiente criptomonedas en caso de que se edite una venta, ya que es una edición cruda sin necesidad de estas comprobaciones.

Pantalla análisis del estado actual

Frontend

Necesitamos una vista en donde podamos ver el estado actual de nuestras finanzas. Vamos a hacerlo mostrando:

- Las criptomonedas que tenemos, su cantidad, y cuánto dinero representa.
- El monto total de dinero que tenemos.

¿Cómo lograrlo? Veamos un ejemplo.

Supongamos que tenemos las siguientes transacciones en la base de datos (acá lo representamos con un JSON solamente para explicarlo):

```
[
   {
       "_id": "60eb148da4666761000216f9",
       "crypto_code": "usdc",
       "crypto_amount": "1.01",
       "money": "165.23",
       "action": "purchase",
       "datetime": "2021-11-07T17:50:00.000Z"
   },
   {
       "_id": "60eb149ba4666761000216fc",
       "crypto_code": "usdc",
       "crypto_amount": "1.01",
       "money": "170.98",
       "action": "sale",
       "datetime": "2021-11-07T20:50:00.000Z"
   },
```

```
    {
        "_id": "60eb148da4666761000216f7",
        "crypto_code": "bitcoin",
        "crypto_amount": "0.01",
        "money": "58447",
        "action": "purchase",
        "datetime": "2021-11-11T17:50:00.000Z"
    },
    {
        "_id": "60eb148da4666761000216f5",
        "crypto_code": "bitcoin",
        "crypto_amount": "0.02",
        "money": "116894",
        "action": "purchase",
        "datetime": "2021-11-12T17:50:00.000Z"
    }
]
```


Con eso podemos deducir que:

- No tenemos USDC, debido a que aunque compramos 1.01, luego vendimos la misma cantidad.
- Tenemos 0.03 Bitcoins (ya que sumamos la compra de 0.01 y la de 0.02).

Asimismo, usando la API de https://criptoya.com/, podemos obtener el precio del bitcoin al momento actual (recordar que para entender cómo funciona esta API, se debe acceder https://criptoya.com/api). Si hacemos una consulta a https://criptoya.com/api/satoshitango/btc/ars/, y estos nos devuelve el siguiente cuerpo:


```
{
    "ask": 5912442.48,
    "totalAsk": 5971566.9,
    "bid": 5797256.86,
    "totalBid": 5739284.29,
    "time": 1626027655
}
```


Con todos estos datos (que tenemos 0.03 Bitcoins después de calcularlo, y que 1 Bitcoin valen $5.739.284,29), el backend debería devolver la información para mostrar la pantalla con la siguiente información:

Criptomoneda	Cantidad	Dinero
Bitcoin	0.03	$ 172.178,53
Total		$ 172.178,53

USDC no se muestra debido a que no tenemos más (compramos pero vendimos después).

Entrega

Se debe notificar al email de el/los profesor/es que el trabajo ha sido entregado. El mismo debe ser subido como mínimo 2 días anteriores a la mesa de examen en el cual será evaluado (dicho tiempo será usado para estudiar el sistema hecho).

Emails:

- Comisión 14:00 hs:
  - yayamgr@gmail.com
- Comisión 19:00 hs:
  - agustinruatta@gmail.com
  - ferdcardona@gmail.com

Uso de sistema de control de versiones

El código fuente deberá ser administrado usando Git, y subido a un servidor de Git como GitHub o Bitbucket. Se valorarán las buenas prácticas como el uso de ramas dentro de Git.

No se permiten entregas de comprimidos al email. Siempre debe estar en un repositorio Git.

Mejoras a la aplicación

Aunque compleja, esta aplicación está lejos de poder usarse en un ambiente profesional. Por ende, se le pueden hacer mejoras que **no son obligatorias**, pero sumarán puntaje extra a la nota final (es decir, no son necesarias hacerlas sino opcionales, pero si sentís que querés aprender más, ¡es ideal que se hagan!).

Dificultad Media: Uso de tests

Una aplicación sin testing unitarios y de integración no puede considerarse robusta. Incluso se pueden usar varios enfoques como TDD.

Investigar cómo se introducen tests en Vue.js y en Javascript, y realizarlo.

Dificultad Media: Agregar gráficos

Usando alguna librería Javascript, agregar gráficos para, por ejemplo, mostrar a través de un gráfico circular la composición de mi cartera de inversión.

Dificultad Media: ¿dónde comprar o vender?

Como se puede ver en la página de https://criptoya.com, cada exchange tiene un precio de compra y venta diferente. Es por eso que se le podría agregar a la aplicación una pantalla en donde se selecciona la criptomoneda, si se desea comprar o vender, y busque entre todos los exchange que lo trabajan para ver en cuál de todos conviene realizar la operación (ya que te pagan más en caso de vender, o te sale más barato en caso de comprar).